PES UNIVERSITY

CELEBRATING 50 YEARS

Data Structures

Dilip Kumar Maripuri

Computer Applications

PES UNIVERSITY

Data Structures

Session : Binary Search Trees - Deleting a node in a BST

Dilip Kumar Maripuri
Computer Applications

Data Structures
Deleting a Node in BST

PES UNIVERSITY

- When deleting a node from a BST, there are **three cases to handle**:
  1. **Node to be deleted is a leaf node (has no children)**
     - Simply remove the node and free its allocated memory.
  2. **Node to be deleted has only one child**
     - Replace the node with its child.
     - Free the allocated memory for the node.
  3. **Node to be deleted has two children**
     - Find the **inorder predecessor** (largest node in the left subtree) or **inorder successor** (smallest node in the right subtree).
     - Replace the node's value with the inorder predecessor/successor's value.
     - Recursively delete the inorder predecessor/successor.

Data Structures

Deleting a Node in BST

Algorithm 3 Delete Node from BST

```
1:  function DELETE(root, key)
2:      if root = NULL then
3:          Display "Item not found in tree"
4:          return root
5:      end if
6:      if key < root.data then
7:          root.left ← DELETE(root.left, key)
8:      else if key > root.data then
9:          root.right ← DELETE(root.right, key)
10:     else
```

Deleting a Node in BST


```
11:     if root.left = NULL and root.right = NULL then        ▷ Leaf Node
12:         Free root
13:         return NULL
14:     else if root.left = NULL then                         ▷ One Child (Right)
15:         temp ← root
16:         root ← root.right
17:         Free temp
18:     else if root.right = NULL then                        ▷ One Child (Left)
19:         temp ← root
20:         root ← root.left
21:         Free temp
```

Deleting a Node in BST


```
22:         else                                              ▷ Two Children
23:             temp ← FindLargestNode(root.left)
24:             root.data ← temp.data
25:             root.left ← Delete(root.left, temp.data)
26:         end if
27:     end if
28:     return root
29: end function
```

Algorithm 4 FindLargestNode(root)

```
1: function FindLargestNode(root)
2:     if root = NULL then
3:         return NULL
4:     end if
5:     while root.right ≠ NULL do
6:         root ← root.right
7:     end while
8:     return root
9: end function
```

Data Structures
Deleting a Node in BST

- **Delete a Node**
  - If the node is **not found**, print an error message.
  - If the node is a **leaf**, simply delete it.
  - If the node has **one child**, replace it with the child.
  - If the node has **two children**, replace it with the largest node in the left subtree (inorder predecessor) and delete the duplicate.

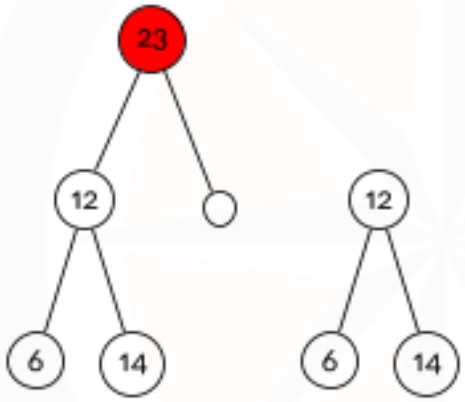
- **Find largest Element Location**
  - If the tree is empty (`NULL root`), return `NULL`.
  - Start from the given `root`.
  - Move **right** until reaching a node with `NULL` right child.
  - This node is the largest in the given subtree.
  - Return this node.

```
1  NODE deleteNode(NODE root, int key) {
2      if (root == NULL) {
3          printf("Item not found in tree\n");
4          return root;
5      }
6      // Step 1: Locate the node
7      if (key < root->data)
8          root->left = deleteNode(root->left, key)
               ;
9      else if (key > root->data)
10         root->right = deleteNode(root->right,
               key);
11     else {
```

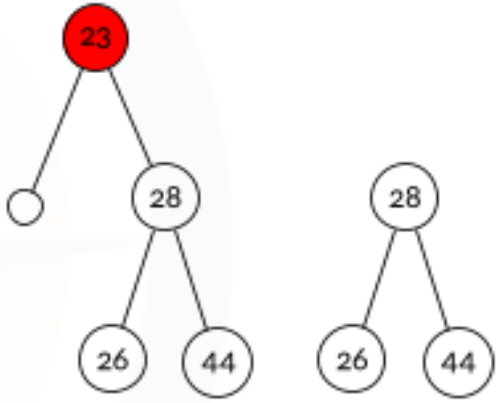
```
1        // Node found: Handle different cases
2        // Case 1: Leaf node (no children)
3        if (root->left == NULL && root->right ==
              NULL)
4        {    free(root);
5                return NULL;     }
6
7        // Case 2: One child (right child exists
             )
8        else if (root->left == NULL) {
9             NODE temp = root;
10            root = root->right;
11            free(temp);              }
```

```
1          // Case 3: One child (left child exists)
2          else if (root->right == NULL) {
3              NODE temp = root;
4              root = root->left;
5              free(temp);                    }
6          // Case 4: Two children (replace with
               inorder predecessor)
7          else {
8              NODE temp = findLargestNode(root->
                   left);
9              root->data = temp->data;
10             root->left = deleteNode(root->left,
                   temp->data);                }
11     }
12     return root;
13 }
```

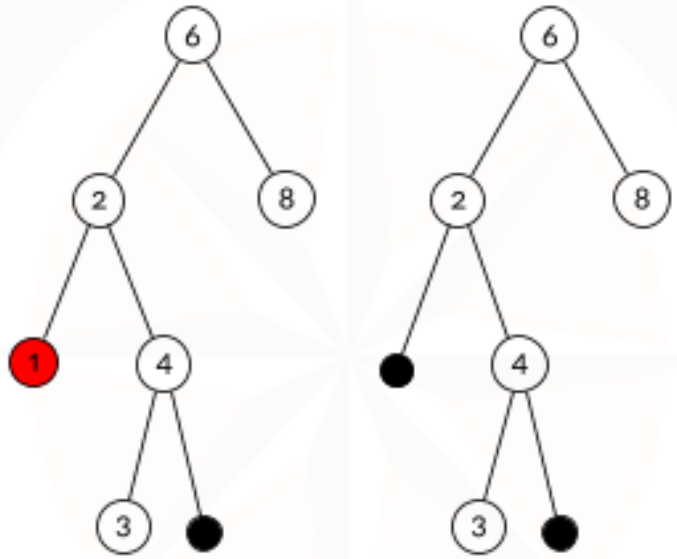
Direct Delete



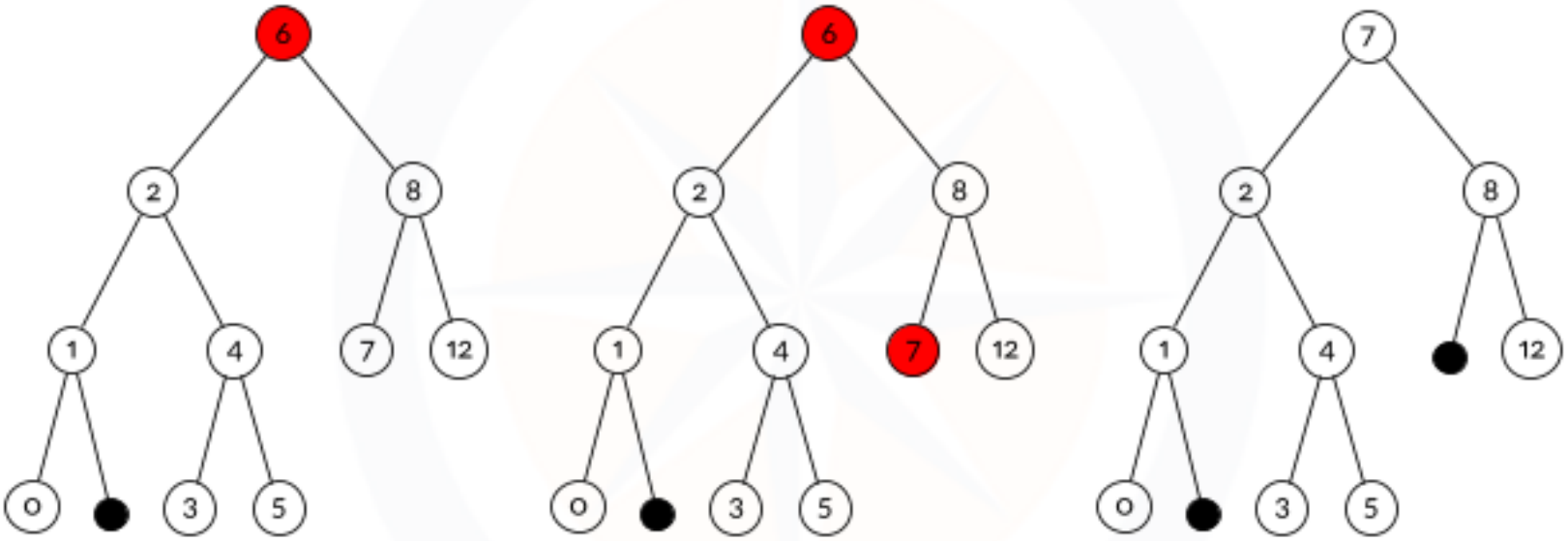
Only Left Child



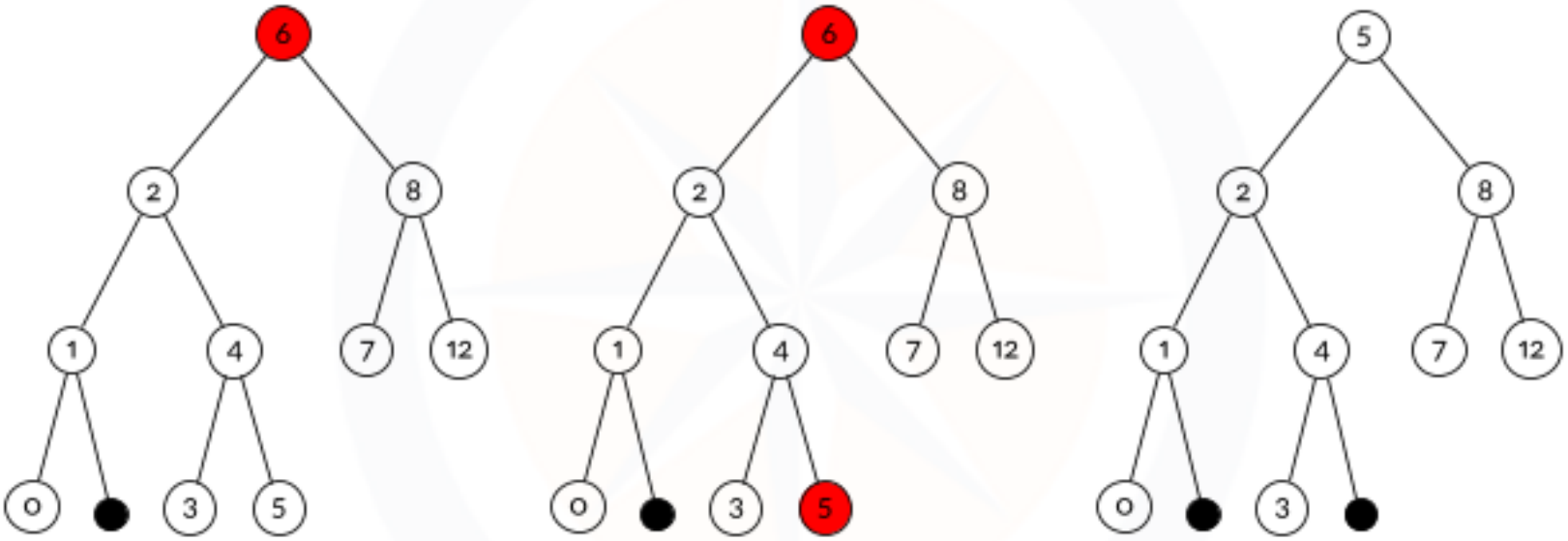
Only Right Child



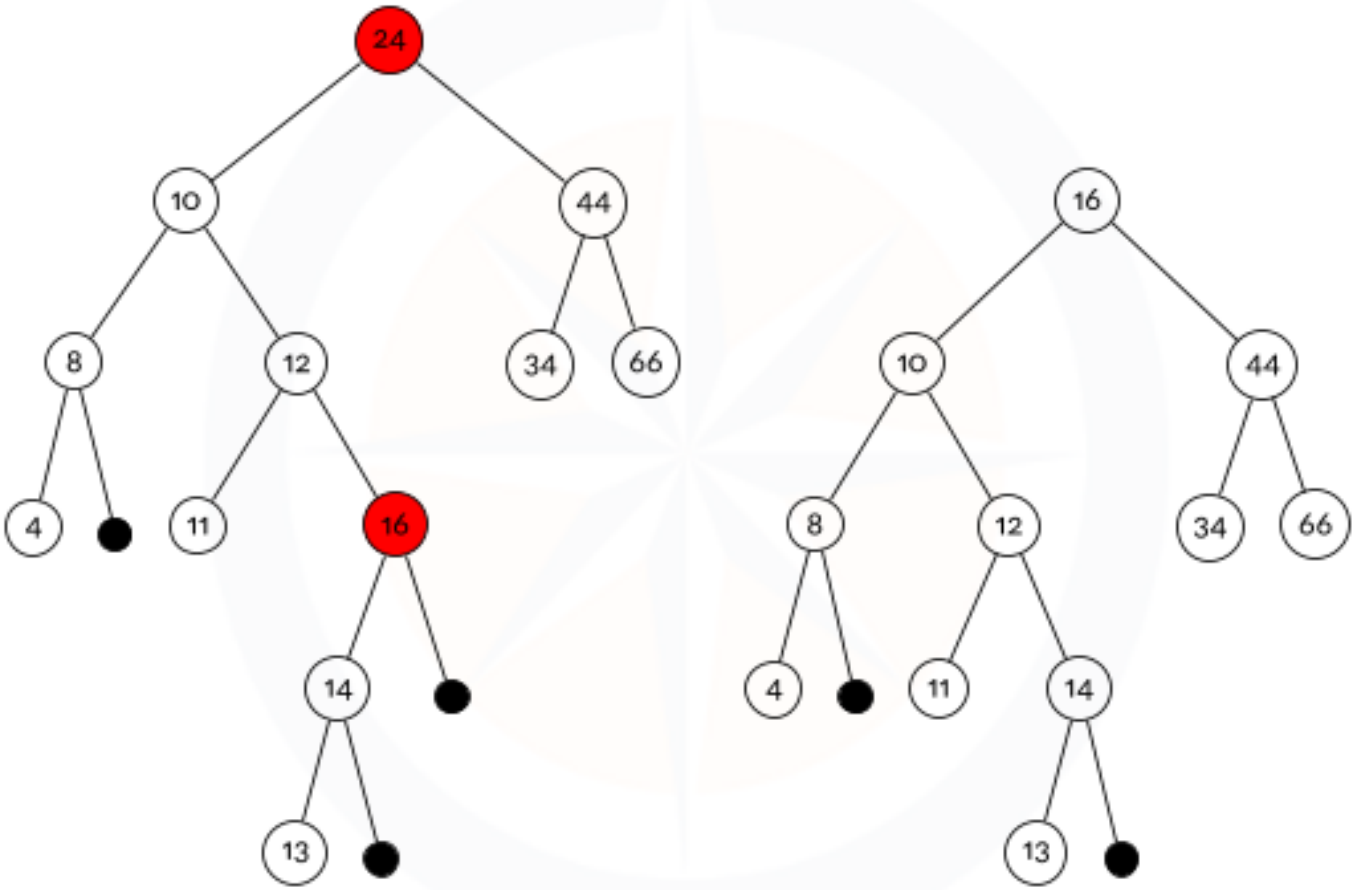
Leaf Node



Case : Both Children $->$ Inorder Successor Method



Case : Both Children $->$ Inorder Predecessor Method


24
10
44
8
12
34
66
4
11
16
14
13
16
10
44
8
12
34
66
4
11
14
13

Data Structures

Searching for a Node in BST

PES UNIVERSITY

- The **search operation** in a **BST** is based on its properties:
  - **If the key is smaller** than the root, search in the **left subtree**.
  - **If the key is larger**, search in the **right subtree**.
  - **If the key is found**, return the node.
  - **If the key is not found**, return `NULL`.

Data Structures

Searching for a Node in BST

Algorithm 5 Search in BST

```
1: function SEARCH(root, key)
2:     if root = NULL or root.data = key then
3:         return root
4:     end if
5:     if key < root.data then
6:         return SEARCH(root.left, key)
7:     else
8:         return SEARCH(root.right, key)
9:     end if
10: end function
```

```
1  NODE search(NODE root, int key) {
2      if (root == NULL || root->data == key)
3          return root;
4      return (key < root->data) ? search(root->
           left, key) : search(root->right, key);
5
6  }
```

Data Structures

Performance of BST Algorithm

Operation	**Balanced BST ($O(\log N)$)**	**Skewed BST ($O(N)$)**
Search	$O(\log N)$	$O(N)$
Insertion	$O(\log N)$	$O(N)$
Deletion	$O(\log N)$	$O(N)$

Table: Time Complexity of BST Operations

Data Structures
Performance of BST Algorithm

PES UNIVERSITY

- **Height of a BST**
  - **Balanced BST:** $O(\log N)$
  - **Worst Case (Skewed BST):** $O(N)$
  - **Height of a Perfect BST:**

$$h = \log_2(N+1) - 1$$

  - **Height of a Skewed BST:**

$$h = N - 1$$

Performance of BST Algorithm

- **Number of Nodes in a BST** For a BST of height h:
  - **Minimum Nodes (Skewed Tree):** $h + 1$
  - **Maximum Nodes (Full BST):**

$$N = 2^{(h+1)} - 1$$

Performance of BST Algorithm

Aspect	Balanced BST	Skewed BST
Structure	Well-distributed nodes	Nodes mostly on one side
Height	$O(\log N)$	$O(N)$
Search Time	$O(\log N)$	$O(N)$
Insertion / Deletion	$O(\log N)$	$O(N)$
Example	AVL Tree, Red-Black Tree	Linked List-like Tree

Table: Comparison of Balanced and Skewed BSTs

Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026